

LATTICE · THE OFF SWITCH

Some decks want none of this.

One line of front matter, and every backtick span goes back to being text.

Why an off switch exists.

The pill grammar readseverysingle-backtick span in every deck.

Zero collisions measured across our own 12,551 — but that is *our* corpus.

A deck written elsewhere may say `[x]` in its prose and mean the characters.

Escaping is right for one span and absurd for ninety.

Nothing on this slide is drawn.

01 A pill that isn't Would be a capsule `{STABLE}:c2`

02 A mark that isn't Would be a green disc `[x]`

03 A half mark that isn't Would be an amber disc `[-]`

04 Ordinary code, unchanged either way Always literal `getId()`

Two values, two behaviors.

RICH – THE DEFAULT

Set `inline-code: rich`, or omit the key entirely. `{STABLE}:c2` draws a capsule and `[x]` draws a disc. Every deck we ship renders this way.



LITERAL – THIS DECK

Set `inline-code: literal`. `{STABLE}:c2` and `[x]` are characters. Nothing else changes: same theme, same layouts, same everything.

The value is `literal`, not `off`.

`inline-code: off` is **not** a known value — the deck keeps drawing pills.

`lint:deck` warns `unknown-inline-code` and names the two real values.

The Studio writes the canonical value for you: **General · Inline pills and marks.**

Unknown always falls to the *running* default, never to silence.

The header up there is the proof.

Chrome is markdown too — header: and footer: render inline like a paragraph.

So a literal deck has to silence them as well, or a foreign deck's running header still draws.

Look at the top of every page in this deck: {DEMO} and [x], as characters.

One slide at a
time:

```
<!-- _class: inline-code-literal --> scopes it without the  
> register.
```

On a raw Marp deck, use Marp's own directive.

In the VS Code Marp preview the sandbox blocks reading the deck's `.md`.

So the **class** is the contract, not the register.

`class: inline-code-literal` in front matter — marp-core stamps every section.

Careful: a slide's own `_class:` replaces the global one, and the grammar returns.

Your text, as you typed it.

*\[x] keeps its backslash here — with no
grammar running, there is nothing to escape.*